

A Survey on the Effectiveness of the Secure Software Development Life Cycle Models

Jing-Chiou Liou* and Saniora R. Duclervil
School of Computer Science and Technology
Kean University

Email: jliou@kean.edu and duclersa@kean.edu

Abstract

Today, a central and critical aspect of cybersecurity problems are related to software problem. Software security is about the understanding of software-induced security risks and how to manage them. To manage software security effectively, we need to understand the process of designing, building, and testing software for security. The System Development Life Cycle (SDLC) process that is currently used to support software development does not address any security components until after the software is developed. From the perspective of software security, the Secure Software Development Life Cycle (SSDLC) is similar to the SDLC but includes security components in its phases. There have been many SSDLC models proposed that are primarily modified from preexisting SDLC models. A study was conducted to survey a selected group of SSDLC models and their effectiveness. The authors first identified four popular SSDLC models used in the IT industry, and then analyzed their common characteristics to derive four sets of criteria for comparison. These criteria are: Focus Areas of Application, Implementation of Model, Security Implementations and Enhancements, and Security training and Staff. Overall, the comparison results demonstrate that the Rastogi and Jones model is considered to be an effective one for many IT projects, especially for Agile projects. However, it is worthy to mention that, because of the various types of IT projects, one specific model cannot be applied for use in all types of IT projects. For an IT project operated in Waterfall, the BSI Seven Touchpoints model can be an excellent alternative.

Keywords

System Development Life Cycle (SDLC), Secure Software Development Life Cycle (SSDLC), Cybersecurity, IT Project Management.

1 Introduction

Cybersecurity has become prevalent and important in today's society. Some recent cybersecurity incidents have made many people victims of cybercrimes. For an example, Equifax underwent a data breach in 2017 [9], and over 143 million Americans became victims of the data breach. Information stolen included credit card numbers and social security numbers. In 2019, 2 billion records were exposed in a massive smart home device breach with the Chinese-based company, Orvibo [15]. In December 2019, a home security camera company, Ring, suffered from a security breach that leaks sensitive information belonging to over 3600 customers [24]. There are also reports that the Ring security cameras were hacked and controlled by cyber predators [24].

Two primary factors contribute to the modern cybersecurity problems. One is the connectivity, and another is the software. In the Internet era, almost every computing device connects to some sort of networks, and eventually connects to the Internet. These Internet-connected devices provide convenient applications and services, and have become an integrated part of our lives. Disconnecting from the Internet to reduce the cybersecurity risks is not a realistic option for many.

Therefore, to tackle the cybersecurity issues, we have to focus on software security. Software security is about understanding the software-induced security risks and how to manage them. To manage software security well, we need to comprehend the process of designing, building, testing software for security.

2 SOFTWARE DEVELOPMENT LIFE CYCLE

Information Technology project management uses a process to develop a software application by planning, designing, executing and monitoring that application in order to meet organizational needs. This type of process or model is called the System Development Life Cycle or Software Development Life Cycle (SDLC). The Software Development Life Cycle is a framework that defines the process used by organizations to build an application from its inception to decommission. The SDLC can take months or years to complete.

There are quite a few different models of SDLC proposed and used in the IT industry. The two most popular ones are Predictive and Iterative. Based on their characteristics and the technology advancements, each is used in industry sectors in the past and current time. Regardless of the type of SDLC models, most divide the development process into phases, or stages through which software is developed. The objectives of each phase are to:

- Define the work items and steps that will be performed
- Produce deliverables
- Ensure the deliverables meet the quality criteria

At the end of each phase, a quality gate or milestone is used to verify the completion of all required works and deliverables are produced as expected. For an example, at the end of Requirement Analysis phase, before the project can move into System Design phase, A Software Requirement Documents (SRD) will be reviewed and baselined. Project stakeholders, including project team and end users or customers, will then sign-off a quality gate document that indicates officially the end of such phase. A successful review of the quality gate marks the time to move into next phase.

2.1 Predictive Model

The predictive life cycle is used when the scope of the project can be clearly articulated in advance of the project beginning, and the schedule and cost can be predicted. In the early era of computing, this was a feasible and preferable process to develop a software: As this model focuses on concept planning and the design of software before its implementation, the predictive process model provides more confidence to the software development team on the success of software project.

One of the most widely used model in this category is the Waterfall. The Waterfall process was initially proposed by Benington [4] in 1956 and later refined by W. W. Joyce in 1970 [25]. In his original proposal, there are seven phases in the process:

- Systems Requirements
- Software Requirements
- Analysis
- Program Design
- Coding
- Testing
- Operations

However, through the evolution in software engineering, variations in the phases were developed in the past. As shown in Figure 1, most recent accepted version combines both “Systems Requirements” and “Software Requirements” into a single “Requirements” phase. The “Analysis” and “Program Design” were also integrated into just a “Design” phase. Some change the “Coding” phase into an “Implementation” or

“Development” phase. People also use “Verification” in place of “Testing.” Finally, the “Deployment” phase is also added into the process to distinguish the efforts taken to install the software into the production environment.

The Waterfall model was used heavily in the late 20th century in the software industry. There are other variations of Waterfall models. This rigid, linear, and inflexible process has declined in its popularity to the more adaptive process models in the early 21st century. Nonetheless, it is still used by some of the software projects that are not web applications today.

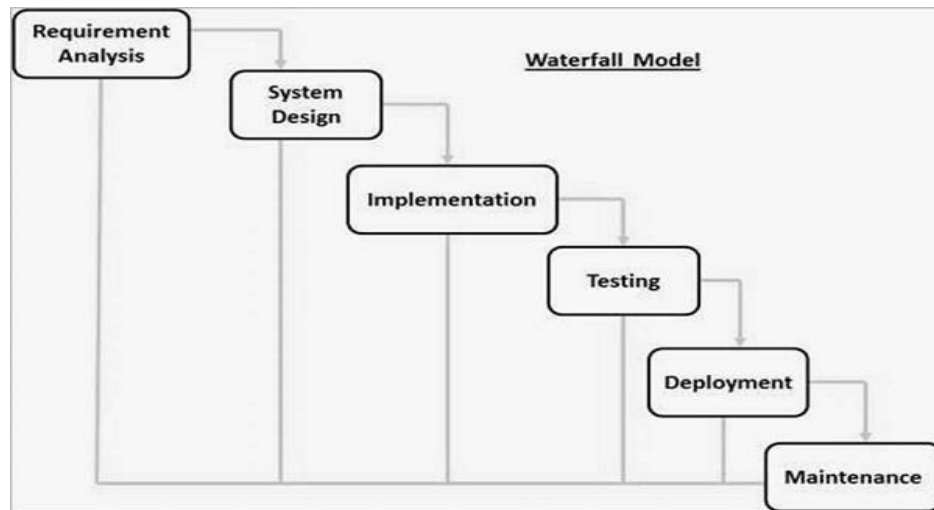


Figure 1 Waterfall Model

2.2 Iterative Models

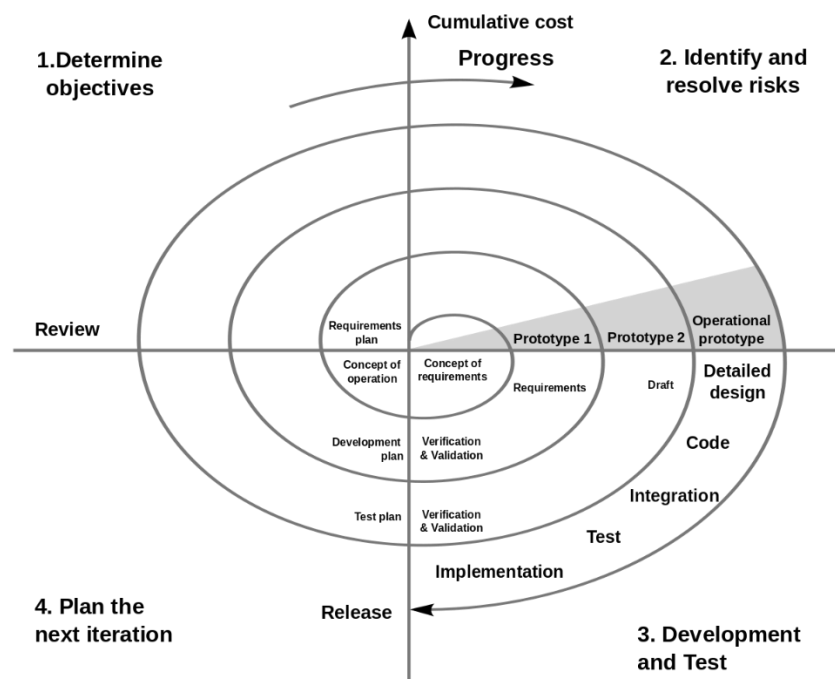


Figure 2 A Spiral Model

There are many process models proposed under the category of iterative SDLC. One that is getting attention is the Spiral model proposed by Boehm in 1988 [5]. The Spiral model, as shown in Figure 2, is a meta-life cycle model because many iterative SDLC models proposed later are based on the same concept as the Spiral model. A metamodel is a model of a model, and metamodeling is the process of generating such metamodels. In each iteration, a prototype is developed and reviewed using a Waterfall-like phases. The review results are used to improve on the prototype in the next iteration. The iteration continues until the software meets user expectations.

Although the Spiral is considered to be a very good process model, it is never used by the software industry: During the time of the client-server architecture, using the Spiral requires contract flexibility and the developers being able to assess any project risks to avoid failure caused by constantly changing tasks. As the phases in each iteration are similar to those in the Waterfall, the Spiral seems just like using Waterfall to develop multi-releases of software.

One iterative process model that later became the dominate SDLC in the software industry is the Agile model. J. Sutherland and K. Schwaber first conceived the Scrum as a set of processes at OOPSLA '95 [26]. Both, along with 15 others, later initiated with the same concepts of Scrum in 2001 the term “Agile” in the “Manifesto for Agile Software Development [3].

The Agile model uses an iterative spiral SDLC but extends the advantages of an iterative model by rapid delivering of software. Agile divides the software into small builds and encourages developing these builds in iterations, called Sprints. As each build consists of only a small set of functionalities, a sprint is usually timeboxed to take two to four weeks to complete.

In addition to Scrum, there are different subtypes of the Agile process models. These subtypes of the Agile process model include Rapid Application Development (RAD) [13], Prototyping [27], Rational Unified Process (RUP) [10], Extreme Programming (XP) [2]. However, we will simply use the umbrella term Agile as a type of SDLC in the comparison.

In the Agile model, especially in Scrum, it is noted that the software requirements are presented in the form of 'User Stories'. User stories describe the use scenarios or the user behaviors which need to be supported by the software. A user story describes the type of user, what they want and why. Hence, a user story is intended to capture a description of a software function from an end-user perspective.

After carefully planning and designing the user stories at the Initiate stage, those user stories are allocated into different Sprints. During a Sprint, the allocated set of user stories are developed, tested and released into the production system. Once the Sprint is complete, the life cycle proceeds into next Sprint with another set of user stories. Each user story forms a software module and the development of the module follows a streamlined Waterfall process.

3 Secure Software Development Life Cycle

Regardless of the specific approach, whether Waterfall or Agile, the SDLC is the process that IT project managers and teams use to develop a software. However, security components are conventionally not incorporated into the SDLC process. It is when the software is completed and operational that security is considered. Hence, this leaves room for vulnerabilities and for hackers to attack the system.

To enhance security features, in the past it was common practice to perform security-related activities only as part of testing. However, most testing is performed based on the functionalities or features described in the software requirements or user stories. Therefore, there is a need to add security-related functionalities or features into requirements or user stories. However, challenges present while using this practice.

The first challenge is that how many security requirements are actually needed for the software to be considered secure. There could be a false sense of software security when many security requirements are implemented. Moreover, most of the security requirements, such as prevention of SQL injection and session

hijacking, are not user requested functions. Should the software company implement a software with such security features at its own cost? In a 2017 EU public opinion survey, most respondents (61%) say they consider security and privacy features when choosing an information technology product and only more than one quarter (27%) say they are ready to pay more for better security and privacy features [7]. In another PwC US Protect.me survey, it shows that 85% will not engage with a company if they have concerns about its security practices [23]. Therefore, a software company needs to balance the number of security features for its software and the cost to developing such features at its own expense.

That is when the Secure Software Development Life Cycle (SSDLC) comes in to fill the gap in recent secure software development practices. The Secure Software Development Life Cycle usually follows the same process as SDLC that the organization adopted and it also has the same phases. However, in this case, security is incorporated in each phase of the SSDLC. The only problem is that, just like the SDLC, the SSDLC is not one size fits all approach. There are many new SSDLC models which have been proposed or modified from existing SDLC models and not all models can work for all types of IT projects because of the different needs and specifications.

Most of the proposed SSDLC models are developed with additional activities or components inserted in the corresponding SDLC. Some SSDLC are proposed primary based on the Waterfall modes [11, 12, 14, 16, 19, 20]; others target the Agile model [1, 6]. To find out how effective a SSDLC will work for most or all IT projects, we have to understand the fundamental characteristics of many proposed SSDLC models. A basic Waterfall-based Secure Software Development Life Cycle model, depicted in Figure 3, is provided below as a reference for discussion.

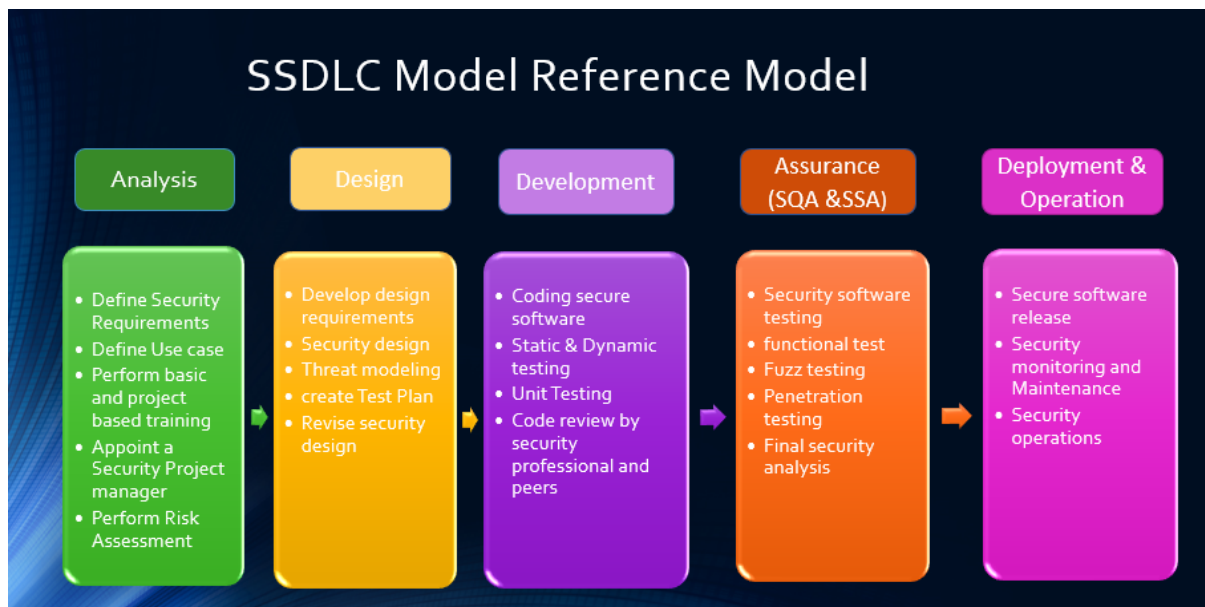


Figure 3 SSDLC Reference Model

Under each phase, we have included some of the common and necessary components for an effective SSDLC:

- Analysis: Among the activities in the phase, performing risk assessment is considered a more effective and leading technique in the phase.
- Design: Threat modeling will help for secure software design.
- Development: Adopt best practice in secure coding, such as using static code analyzer and code review will be more effective.

- Assurance: For SSDLC, adopting a software assurance approach, instead of just testing, provides more benefits to the process. It consists of activities for penetration test and risk-based test.
- Deployment and operation: Secure operation practice is usually ignored in the SSDLC

4 Analysis of SSDLC Models

There are several comparisons of different SSDLC models from other researchers [8, 28, 29]. However, these articles do not perform actual comparison based on any measurements or use criteria derived from the characteristics embedded in the SSDLC models that they compare, such as if additional training is required for the SSDLC. They simply list the SSDLC models and described the security related activities/components inserted into the original SDLC.

To perform the comparison of SSDLC models in more technical depth, we need to look into the characteristics of those popular models and select the common characteristics across the models for comparison. To the end, we have to firstly determine the SSDLC models to compare.

There are many SSDLC models proposed in the last two decades: Some are used by certain software organizations, and some later became a recommended process and were partially or completely adopted into standards. For the comparison, we have carefully selected some of the popular models from them. The first model considered is Microsoft's Trustworthy Computing Security Development Life Cycle model [14, 18, 19]. The Microsoft's Trustworthy Computing Security Development Life Cycle model, or the SDL (see Figure 4) as Microsoft calls it, was developed and adapted by Microsoft Corporation in 2004 [14] for internal use and as a way to prevent any vulnerabilities in the development of applications. This model has a total of six phases: Requirements, design, implementation, verification (which is the testing phase), release, and support and servicing. Most security components are the same as the basic SSDLC, but Microsoft has a security Kick-off meeting in the first phase, and the need to register with Microsoft's SWI (Secure Windows Initiative).



Figure 4 Secure Software Development Process Model at Microsoft©

The second model is the Seven Touchpoints Model [16]. This model was developed by Gary McGraw in 2004 and the highlights involve software security touch points, which is also known as 'best practices.' As shown in Figure 5 [17], it is later called "Building Security In" and is adopted by some organizations, such as the Software Engineering Institute (SEI). This model has 6 phases: Requirements and use case, Architecture and design, Test plans, code, Tests and Test results, and Feedback from the field. The security components are more or less the same but the difference is that the Touch Point model is a Waterfall and an Agile development model combined to make one model.

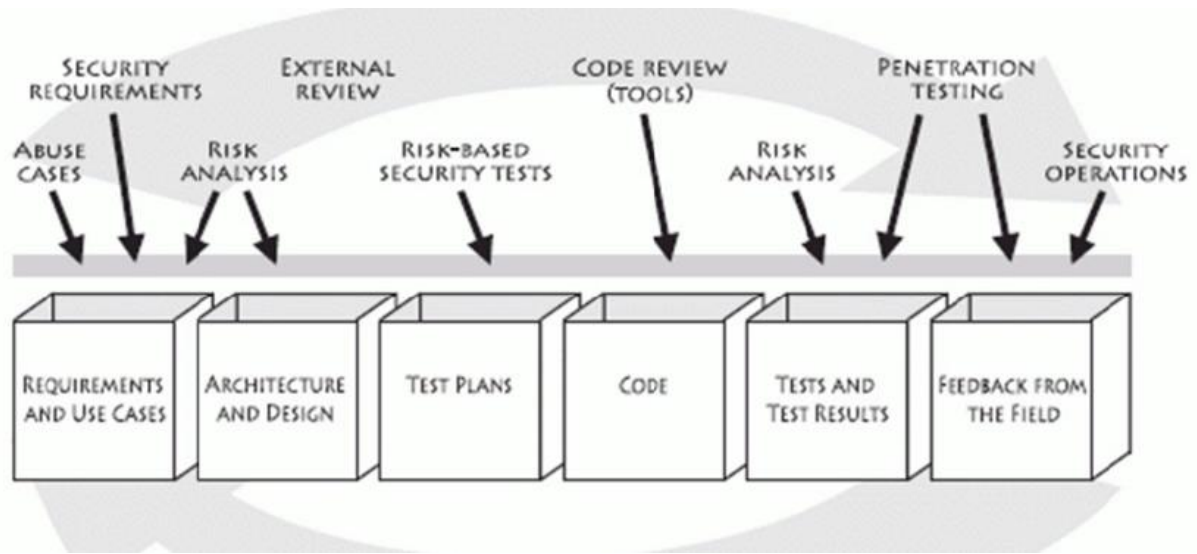


Figure 5 Building Security In Seven Touchpoints Model

The Software Engineering Institute (SEI) Team Software Process (TSP) for Secure Software Development [21, 22] is chosen for the third process model. This SSDLC was developed by SEI. The Software Engineering Institute developed this SDLC model that ties with the Team Software Process (TSP) and Personal Software Process (PSP). It aims to decrease the likelihood of appearances of vulnerabilities in software developed. Shown in Figure 6, this model has only four phases which is not the same for the basic SSDLC reference model shown in Figure 3. It has a requirement, design, implementation, and testing phase but no operation and deployment phase. Organizational policies, management oversight, resources and training, project planning, project tracking, risk management, measurement and feedback are incorporated into all of the phases. It is a more thoughtful, but also a very complicated model.

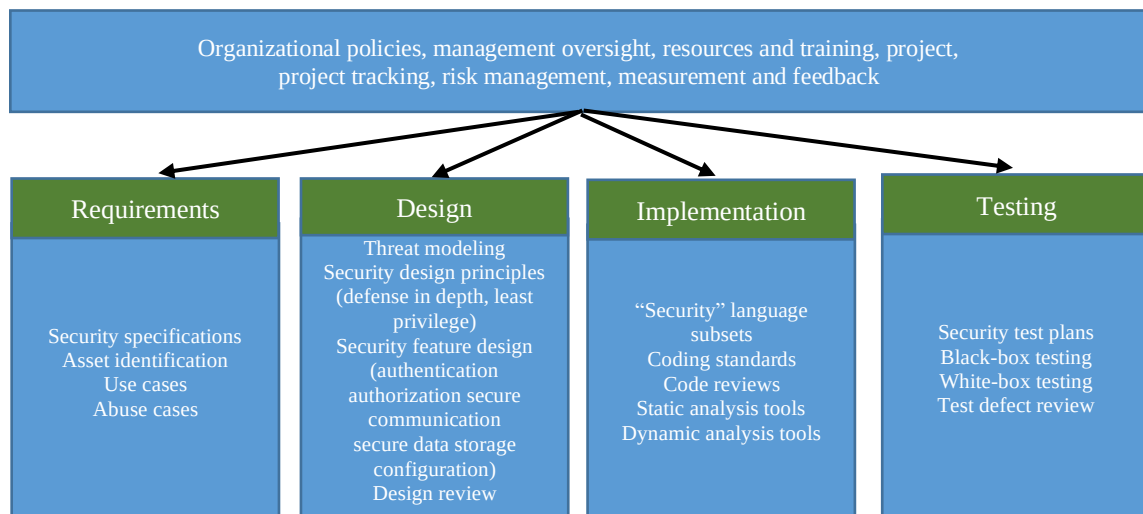


Figure 6 SEI TSP for Secure Software Development

The last model identified is the SSDLC model developed by Jones and Rastogi [11]. This model, as we can see in Figure 7, has a total of five phases: Design, development, Testing, Operation and Maintenance, and the Disposal phase.

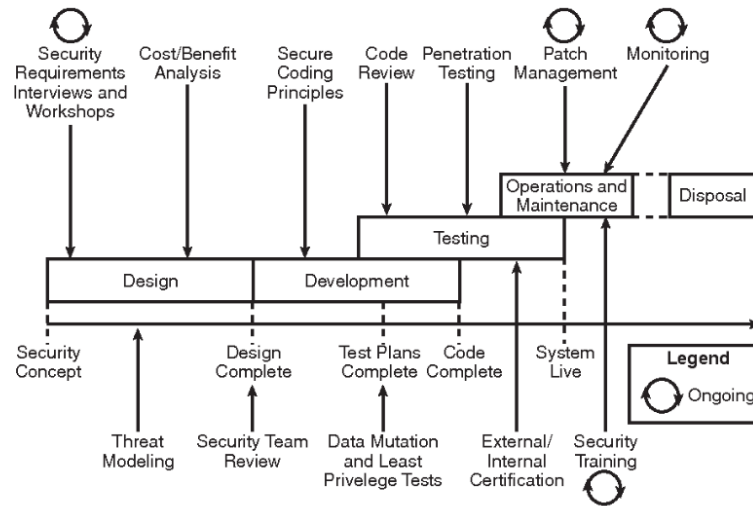


Figure 7 Jones and Rastogi's SSDLC

Compared with the other three, there are few differences in this model. This model has no separate requirement phase. The requirement gathering and workshops are done in the design phase. Training is not provided to the development team, but to the end user in the operation and maintenance phase and is ongoing even after the process. Certification is required at the end of the testing. There is a disposal phase that focuses on if and when the program is being disposed of, moved, remodeled, or archived. This last phase is a plus because there are security protocols and components that need to be added when disposing of, the redeveloping of or moving the software.

5 Analysis of Characteristics and Assessment Criteria

To perform the comparison for the four models of SSDLC, we started with studying each model and observing significant characteristics from each. Once all information was collected after studying all models, we were able to identify some characteristics that all models or some of them have in common.

Based on those similar characteristics that concluded from the study of the four models, we have developed four sets of criteria: Focus area of Application, Implementation of model, security Implementations and Enhancement, and Security training and staffing.

5.1 Focus process Areas of Application

This group focuses on the process area of the application or the software, as to whether it fits well for Agile, Waterfall or both. Moreover, its relevance to a particular industry sector is also examined, and if it contains all phases.

The first criterion identifies if the model is either Waterfall or Agile rooted. When a model is based on Waterfall, this is an indication that the project is going to last few months to few years. When a SSDLC model is originally targeted on an Agile model, this is an indication that each sprint in the project will take the most a month to complete. Because the scope is much clear in Waterfall, while Agile is more of risk-driven, a SSDLC, or a SDLC model for that matter is more effective when it is applied in either Waterfall or Agile, but not both. The measure of this criterion uses a binary measurement which is yes or no. In this criterion, it determines if the SSDLC models would work well in a Waterfall approach, Agile approach, or both.

The second criterion in this group is that the model has to be universal for most organizations. For this criterion a 1-5 scale measurement is applied: 1 being specific and 5 being general for public use. The

purpose of this criterion is to find a model that is most effective for the use of most or all Information Technology projects.

The third criterion in this group is that the particular SSDLC model has to contain all the phases that are required in the SSDLC reference model. The phases include: Analysis/requirement phase, design phase, development phase, assurance (SQA, SSA) phase, and deployment and operation phase. This criterion has a scaling measurement that is either yes or no. All phases must be included according to the reference model that is provided earlier in this study.

5.2 Implementation of Model

This set of criteria focuses on the implementation of the model and how the implementation will impact on the application when it is being developed.

The first criterion is that the SSDLC model has to be compatible with the development of the software. It means that the model has to incorporate certain components that will enhance and benefit the development of the software. To that end, a percentage measurement for this criterion is chosen. 100% being that the model is completely compatible with the development of the software. This means that regular as well as security components are added into the process and the process model is able to bring forth the functionality and the security components in an effective manner. If the model is at 75% compatibility, that means that the model satisfies only part of the regular function and security function requirements. This could happen if the model does not include all phases of the SSDLC or if one or more phases are lacking security components. If the model is at 50% compatibility that means that only the regular components are added, but not the security components. If the model is at 49% or below, that means that it does not perform neither the regular or security requirements well.

The Implementation of Model criterion is necessary but can be optional. It relates to if the SSDLC is good for a new or a continued development. This criterion focuses on disposing of the software, or developing a new software entirely or moving the software to a new location. The data involved with the software need to be protected to avoid data breaches or vulnerabilities while disposing of, changing the location of, or developing a new software from the existing software.

The third criterion of the group is that the preliminary/initiation phases have to be effective for the development of the software. In the preliminary phase which is mostly the requirement phase in SSDLC, the requirements analysis for the software is performed. Project managers, security project managers and stakeholders are selected as well. Security training takes place for the team involved with the project as well. In a requirement phase, there can also be user stories developed in Agile development. User stories are tasks that can be manipulated at a later time. Even though it is already completed, it gives the team the opportunity to go back and change the user story or fix a problem that occurred. This is usually true in Agile development, because a Sprint in Agile development last less than one month. This criterion tests if the team is able to redo a sprint, add to it, or correct it.

The measurements provided are highly effective, moderately effective, and not effective. A model's requirement phase is highly effective if it has both a requirement analysis and user requirements in a Waterfall model or if there are user stories in an Agile model, if it has training or workshops (Security training, or a certification or coding lessons) for the project team. Security requirements are usually described in one or more user stories in an Agile model. If a model's requirement phase is moderately effective, it has a requirement analysis and a user story but no training or workshop for the team involved with the project. If a SSDLC model's requirement phase is considered ineffective, that means it has a security requirement phase, but provides no training.

The last criterion requires a SSDLC model to have a deployment/operation phase. At the end of the software development, security components and protocols should be taken into account in order to release the software into production environment and to monitor and maintain the software. The measurement provided for this criterion is yes or no. The measure is yes if it has a deployment/operation phase, or no if it does not have a deployment/operation phase.

5.3 Security Implementation and Enhancements

Criteria in this group focuses on the security components of the SDLC which provides the ground for SSDLC.

The first criterion for this group is that security testing has to be performed in at least two or more phases of the SSDLC. In a SSDLC, there are security components in all of the SSDLC phases. Testing is usually conducted mainly in the implementation (UT/IT), testing (ST) and deployment (UAT) phases. This criterion makes a SSDLC more effective with integrated security testing components, so that the software will be less vulnerable to attack. The measurements provided are: Yes, it has two or more phases that does testing, or no if it does not.

The last criterion for this group is that all phases have to have a security component. That is the whole idea for the Secure Software Development Life Cycle model. Security has to be emphasized throughout all phases to ensure in everything that is being done in SSDLC will ensure the safety of the software. The model is 100% effective if all phases have a security component. And it is 50% effective if it does not.

5.4 Security Training and Staffing

This group focuses on whether there is security staff and how difficult training is.

Security training is very essential in a process model for a secure software development. Security training involves educating the team on vulnerabilities, secure coding, threat modeling, security protocols, etc. This criterion is being measured by its difficulty in a 1-10 scale. 1-3 is easy, 4-6 is moderate, and 7-10 is difficult. Security training has to be easy and understandable. If a type of security training requires a software team to get security certification, they wouldn't be able to concentrate on the development of the software. Security training has to be specific and attainable.

The last criterion is if there requires security staff to oversee all security aspect of a SSDLC. Security staff can make sure all security components in the SSDLC are carried out successfully. Implementing a security staff is the same thing as implementing a staff/team for a regular System development life Cycle. In a SSDLC, a security project manager, stakeholder and software team (such as a tester and a software engineer who is skilled in the cybersecurity field) has to be present. A yes or no measurement is provided for this criterion.

6 Comparison of SSDLC Models

6.1 Comparison from Criteria

Based on information collected, as shown in the Table 1, for the Focus Area of Application group of criteria, no model as of now works well for both Waterfall and Agile development. In the second criteria, the more universal model is the Rastogi and Jones model and Microsoft's. In the third criteria, the only model that does not have all the phases required in the SSDLC is the SEI model. The more effective model is the Rastogi and Jones model, the second effective model is Microsoft's model.

Table 1 Comparison Results from criteria 1-5

SSDLC model	Criterion 1: Agile, Waterfall, or both	Criterion 2: Universal (1-5)	Criterion 3: Model has all phases	Criterion 4: Compatibility	Criterion 5: For new or continued development
Microsoft SDL	Waterfall: Yes Agile: Yes Both: No	Scale of 2	Yes	100%	No
BSI Touchpoints	Waterfall: Yes Agile: No Both: No	Scale of 1	Yes	0%	Yes
SEI TSP	Waterfall: Yes Agile: Yes Both: No	Scale of 2	No	75%	Yes
Jones & Rastogi	Waterfall: Yes Agile: No Both: No	Scale of 3	Yes	100%	No

For implementation and model group, SEI is at 75% compatibility because it does not have a deployment/operations phase that has security components and protocols to protect the software when it is released and being monitored. The Gary McGraw Touchpoints model is at 0% compatibility because as of now, Agile and Waterfall development cannot be combined. In the third criteria two models are the leading: The Rastogi and Jones model has a phase that has new or continued development, and the Touchpoints model because of the fact that it can work with Agile development which allows for continued development. SEI is the least effective in this criterion because it is the only one that does not have a deployment/operations phase. The more effective models in this group of criteria are the Jones and Rastogi model and the Microsoft model.

Table 2 Comparison Results from criteria 6-11

SSDLC model	Criterion 6: Initiation phase effectiveness	Criterion 7: Has Operation & Maintenance phase	Criterion 8: Is security testing done in two or more phases	Criterion 9: Security is integrated in all phases	Criterion 10: Difficulty of implementing security training	Criterion 11: Has security staff
Microsoft SDL	Highly effective	Yes	Yes	100%	Scale of 2	Yes
BSI Touchpoints	Ineffective	Yes	Yes	100%	Scale of 10	No
SEI TSP	Highly effective	No	Yes	75%	Scale of 5	Yes
Jones & Rastogi	Highly effective	Yes	Yes	100%	Scale of 8	Yes

For the Security enhancements and implementation group, as depicted in the Table 2, the more effective models are the Jones and Rastogi model and Microsoft's model. Security testing is done in at least two of the phases for all models. All models have security components in each phase except the SEI model because

it does not have a deployment/operations phase which has security components and protocols releasing and monitoring the software.

For the Security training and staff group of criteria, all models have a security staff except for the Touchpoints model, the models that implemented an easy training are Microsoft's model and the SEI model. The Touchpoints model has no security training. In the Rastogi and Jones model, what is involved in the security training is not specified and it is only implemented in the operations phase and continues on after that. The most effective models in this group are the Microsoft model and the SEI model.

6.2 Summary of the Comparison

Microsoft's model seems an effective model, however, it is customized by Microsoft Corporation and is not meant to be used by the general public. In other terms, this model is less effective for most or all IT projects.

The Gary McGraw's Touchpoints is a model that has the potential to be a Waterfall model but not both Agile. As of now we have not found a model that could work well for both Waterfall and Agile development. If this model were to be used, considering the model leans more towards Agile, there would not be enough time to complete all the phases that is crucial to the successful development of the software because Agile development only takes at most a month in a Sprint while Waterfall could take months to years. As of now, a model like this is not effective for SSDLC development yet alone being used by most organizations for software development.

The SEI team process for Secure Software Development has a great training process for the team members in the requirement phase. However, this model does not have the deployment and operation phase that contains security components and protocols for releasing the software to the public and monitoring it to prevent any attacks. As of now, this model is not as effective.

The model that is most effective in the comparison is the SSDLC model developed by Rastogi and Jones. It contains all of the phases and meets most of the important criteria. It has an operation and maintenance phase, it is universal enough for public use, and the process model is compatible with the all types of software development organization. Although a requirements phase is not added, the requirement gathering and workshops are added into the design phase. Security training is not added until the Maintenance phase but it is ongoing even after the development of the software. There is a disposal phase which is not in the universal model but it does prove useful if the software needs to be moved, redeveloped, or disposed of.

5 Conclusion

Since cyber-attacks are becoming more common and more dangerous and unavoidable, it is important that our software is protected, yet alone the process used to develop our software. There are many Secure Software Development Life Cycle models available to the IT industry. Some of them are very similar to each other. However, since most IT projects are different in terms of sizes, operation objectives, as well as the sectors of industry, the SSDLC model has to be general enough to meet at least some of the criteria for an IT organization's software development.

This study is only meant to compare existing popular SSDLC models and to identify which one is most effective for being used for most or many IT projects and the one that would be most effective in protecting a software from various attacks and vulnerabilities.

Overall, the comparison results demonstrate that the Rastogi and Jones model is considered to be an effective one for many IT projects, especially for Agile projects. However, it is worthy to mention that, because of the various types of IT projects, one specific model cannot be applied for use in all types of IT projects. For an IT project operated in Waterfall, BSI Seven Touchpoints model can be an excellent alternative.

References

- [1] Ayalew, T., Kidane, T., and Carlsson, B., Identification and Evaluation of Security Activities in Agile Projects. 2013 Nordic Conference on Secure IT Systems, pp. 139-153, Ilulissat, Greenland, Oct. 2013.
- [2] Beck, K., Extreme Programming Explained: Embrace Change. Addison-Wesley, 2000.
- [3] Beck, K. etc, Manifesto for Agile Software Development, <https://Agilemanifesto.org/iso/en/manifesto.html>. Last retrieved 2/24/2020.
- [4] Benington, H.D., Production of large computer programs. In Proceedings, ONR Symposium on Advanced Programming Methods for Digital Computers, pp 15-27, June 1956.
- [5] Boehm, B.W., A Spiral Model of Software Development and Enhancement, Computer, pp. 61-72, May 1988.
- [6] Daud, M. I.. Secure Software Development Model: A Guide for Secure Software Life Cycle. In Proc. The international MultiConference of Engineerings and Computer Scientist 2010, Vol. I, Hongkong, March 2010.
- [7] European Commission, Special Eurobarometer 460, Attitudes towards the impact of digitisation and automation on daily life, May 2017. <https://ec.europa.eu/commfrontoffice/publicopinion/index.cfm/Survey/getSurveyDetail/instruments/SECIAL/surveyKy/2160>. Last retrieved 2/24/2020.
- [8] Gr'egoire, J., Buyens, K., Win, B. D., Scandariato, R., and Joosen, W., On the Secure Software Development Process: CLASP and SDL Compared, in Proc. 29th International Conference on Software Engineering Workshops, 2007. https://www.researchgate.net/publication/4261954_On_the_Secure_Software_Development_Process_CLASP_and_SDL_Compared. Last retrieved 2/24/2020
- [9] Identity Theft, <https://www.usa.gov/identity-theft>. Last retrieved 2/24/2020
- [10] Jacobson, I., Booch, G., and Rumbaugh, J., The Unified Software Development Process, Addison-Wesley Professional, 1999.
- [11] Jones, R. L., and Rastogi, A., Secure Coding: Building Security into the Software Development Life Cycle. 29-39. Information Systems Security, Vol. 13, Issue 5, 2004.
- [12] Keary, E., & Manico, J. (n.d.). *Secure Development LifeCycle*. [https://www.owasp.org/images/7/76/Jim_Manico_\(Hamburg\)_-Securing_the_SDLc.pdf](https://www.owasp.org/images/7/76/Jim_Manico_(Hamburg)_-Securing_the_SDLc.pdf). Last retrieved 2/24/2020
- [13] Kerr, J. M.; Hunter, R., Inside RAD: How to Build a Fully Functional System in 90 Days or Less. McGraw-Hill, 1994.
- [14] Lipner, S., The Trustworthy Computing Security Development Lifecycle, in Proc. 20th Annual Computer Security Applications Conference, pp 2-15, Tucson, AZ, 2004.
- [15] Massive smart home breach leads to consumer security concerns. <https://www.idtheftcenter.org/massive-smart-home-breach-leads-to-consumer-security-concerns/>. Last retrieved 2/24/2020.
- [16] McGraw, G., Software Security, IEEE Security and Privacy, pp. 80-83, Vol. 2, Issue 2, 2004.
- [17] McGraw, G., Software Security, Building Security In, <http://www.swsec.com/resources/touchpoints/> Last retrieved 3/24/2020.
- [18] Microsoft. (2009, June 30). Security Development Lifecycle for Agile Development. *Microsoft Security Development Lifecycle*. https://www.blackhat.com/presentations/bh-dc-10/Sullivan_Bryan/BlackHat-DC-2010-Sullivan-SDL-Agile-wp.pdf. Last retrieved 2/24/2020.
- [19] Microsoft. (2012). Microsoft Security Development Lifecycle (SDL) – version 5.2. <https://docs.microsoft.com/en-us/previous-versions/windows/desktop/cc307748%28v%3dmsdn.10%29>. Last retrieved 2/24/2020.
- [20] Morana, M. (n.d.). Building Security Into The Software Life Cycle, A Business Case. <https://www.blackhat.com/presentations/bh-usa-06/bh-us-06-Morana-R3.0.pdf>. Last retrieved 2/24/2020.
- [21] NooPur D., Developing Secure Software, in Secure Software Engineering. *The DoD software Tech News, Volume 8(2)*, pp. 3-7, 2005. <http://www.sis.pitt.edu/jjoshi/devsec/securesoftware.pdf>. Last retrieved 2/24/2020.
- [22] Over, J. W., Team Software Software Process for Secure Software Development, 2002. https://resources.sei.cmu.edu/asset_files/Presentation/2002_017_001_24393.pdf. Last retrieved 2/24/2020.
- [23] PwC Consumer Intelligence Series: Protect.me <https://www.pwc.com/us/en/services/consulting/library/consumer-intelligence-series/cybersecurity-protect-me.html>. Last retrieved 2/24/2020.
- [24] Ring throws customers under the bus after data breach, <https://www.eff.org/deeplinks/2019/12/ring-throws-customers-under-bus-after-data-breach>. Last retrieved 2/24/2020.

- [25] Royce, W. W., Manage the Development of Large Software Systems, Proceedings, *IEEE WESCON* 26, 1970. Pp. 1 – 9. <http://www-scf.usc.edu/~csci201/lectures/Lecture11/royce1970.pdf>. Last retrived 2/24/2020.
- [26] The Scrum Guide, pp. 17 <https://www.scrumguides.org/docs/scrumguide/v2016/2016-Scrum-Guide-US.pdf> Last retrived 3/24/2020.
- [27] Software Prototyping, <https://www.ingsoftware.com/software-prototyping>. Last retrived 3/24/2020
- [28] Tiirik, K., Comparison of SDLC and Touchpoints. https://courses.cs.ut.ee/MTAT.03.246/2013_spring/uploads/Main/essay09.pdf. Last retrived 2/24/2020.
- [29] Win, B. D., Scandariato, R., Buyens, K., Grégoire, and J., Joosen, W., On the Secure Software Development Process: CLASP, SDL and Touchpoints Compared, Information and Software Technology archive, Volume 51 Issue 7, pp. 1152-117, July, 2009